干净搜索聚合器 · 构建方案

版本 v1.0 / 2026-11-05 / 状态：设计定稿，待开工 代号：CleanSearch（工作名，可改） 一句话：聚合多引擎检索 + 无广告无钓鱼的结果页 + 三档相关性分组 + 侧栏 AI 总结 + 书签启动台。

---

0 摘要与决策记录

可行性结论：可行。功能上没有任何一项需要"从零写浏览器内核"。全部需求落在三个可独立交付的层次上：检索聚合层、自有 UI 层、LLM 总结层。

最大分水岭：点开搜索结果之后，页面归谁承载。

选项	形态	能否做页面级拦截 / 广告屏蔽 / 任意网页总结
新标签页打开原站	PWA	只有结果层过滤，页面内广告归原站
浏览器扩展	PWA + Extension	能：declarativeNetRequest 拦截、右键总结、一键加书签
自建容器	Tauri / Electron	能，且体验最接近"自己的浏览器"

iframe 方案作废：`X-Frame-Options` 与 CSP `frame-ancestors` 会拒绝绝大多数站点的嵌入，代理渲染又会破坏登录态与交互。

已确认的技术决策

编号	决策	理由
D1	检索层用自建 SearXNG 做统一入口	一次拿多引擎结果、天然无广告、支持 bang、零 API 费用
D2	商业 API 作为兜底而非主力	上游引擎会 CAPTCHA/限流，需要有付费回路保底
D3	后端用 FastAPI（Python）	抓正文、中文分词、向量化、PDF 解析生态更顺
D4	存储用 SQLite + FTS5	个人/小团队规模零运维，随机读极快
D5	三档分组用"公式打分 + LLM 复核"双层	纯阈值切分会把垃圾塞进弱相关栏
D6	总结走流式 SSE + 强制引用编号	首字延迟可控，且幻觉引用可见可控
D7	书签一键添加走"扩展 / bookmarklet / PWA share"三通道	网页 JS 写不进系统书签栏，这是物理边界

---

1 目标与非目标

1.1 目标

1. 搜索结果页零广告、零钓鱼入口，垃圾站不进主栏。

2. 两页结构：搜索页、书签启动台。
3. 搜索页在保留网页结果的同时，侧边输出 AI 总结，不遮挡结果。
4. 支持对任意网页做结构化总结。
5. 结果按"强相关 / 相关 / 弱相关"三板块分组。
6. 输入后缀即可指定搜索引擎与地区（数分知识点 `g` → Google）。
7. 书签支持一键添加、分组、快速启动、导入导出。

1.2 非目标（本期不做）

- 多用户账号体系与权限（单人使用，预留 schema）。
- 图片 / 视频 / 新闻 / 地图的垂直搜索（架构预留 `type` 参数）。
- 自建爬虫与索引（不做独立索引，只做聚合）。
- 广告拦截规则集的社区维护（直接引 EasyList）。

---

2 系统架构



2.1 可复用的现成基座

项目	定位	可复用部分
Vane（原 Perplexica，2026-03 改名）	SearXNG + LLM 的开源 Perplexity 替代，带引用	检索→抓正文→流式总结的完整链路
Morphic	Next.js App Router + 生成式 UI，后端可切 SearXNG / Tavily / Exa	侧栏总结界面的成熟参考
SearXNG	自建元搜索，聚合 70+ 引擎	整个检索层，直接当内部 JSON API

项目	定位	可复用部分
Whoogle / 4get / LibreY	轻量 Google 前端代理	结果清洗与反广告的实现思路

策略：**第 1 周只写差异化部分**（三档分组、后缀路由、书签启动台、侧栏布局），检索与总结链路先复用。

---

3 技术选型

层	选型	版本	理由
前端框架	Next.js App Router	15.x	RSC + streaming 天生适配"边搜边出总结"
UI	React 19 + Tailwind + shadcn/ui	—	组件齐、可裁剪、无运行时负担
状态	TanStack Query + Zustand	—	服务端缓存与本地 UI 状态分开
后端	FastAPI + Uvicorn	0.11x	异步抓取、流式响应、Python 生态
检索	SearXNG	latest	多引擎聚合、JSON 输出、bang
正文提取	trafilatura（主）+ readability-lxml（备）	—	中文正文提取效果好于 boilerpy3
中文处理	jieba + rank_bm25	—	轻量，无需 ES
向量	bge-m3（本地）或 text-embedding-3-small	—	中英混合查询表现稳
存储	SQLite WAL + FTS5	3.45+	零运维，单文件可备份
LLM	Ollama `qwen3:8b` / OpenAI 兼容端点	—	总结任务小模型够用
容器	Docker Compose	—	一条命令起 SearXNG + API
桌面壳（可选）	Tauri 2 + WebView2	—	Windows 体积小、内存低

---

4 目录结构

```
llq/
├─ docker-compose.yml
├─ .env.example
├─ searxng/
│  └─ settings.yml                  # 开启 json 格式、关 limiter
├─ apps/
│  ├─ api/
│  │  ├─ pyproject.toml
│  │  └─ app/
│  │     ├─ main.py                 # FastAPI 入口
│  │     ├─ config.py               # 环境变量
│  │     ├─ deps.py
│  │     ├─ routers/
│  │     │  ├─ search.py            # /api/search
│  │     │  ├─ summarize.py         # /api/summarize (SSE)
│  │     │  ├─ page.py              # /api/page/*
│  │     │  ├─ bookmarks.py         # /api/bookmarks/*
│  │     │  └─ go.py                # /go 风险中转
│  │     ├─ core/
│  │     │  ├─ query_parser.py      # ⑤ 后缀路由
│  │     │  ├─ aggregator.py        # ② 多引擎并发 + 熔断
│  │     │  ├─ dedupe.py            # ③ URL 规范化 + simhash
│  │     │  ├─ safety.py            # ④ 黑名单 + 风险分
│  │     │  ├─ scoring.py           # ⑤ 打分 + 三档分组
│  │     │  ├─ summarize.py         # ⑥ 总结管线
│  │     │  ├─ extract.py           # 正文提取
│  │     │  └─ llm.py               # LLM 客户端（流式）
│  │     ├─ db/
│  │     │  ├─ schema.sql
│  │     │  └─ repo.py
│  │     └─ prompts/
│  │        ├─ summary_system.txt
│  │        ├─ summary_user.txt
│  │        └─ relevance_judge.txt
│  └─ web/
│     ├─ next.config.ts
│     └─ app/
│        ├─ layout.tsx
│        ├─ page.tsx                # 搜索主页
│        ├─ bookmarks/page.tsx      # 启动台
│        ├─ r/page.tsx              # 结果页
│        ├─ s/page.tsx              # 单页总结
│        └─ components/
│           ├─ SearchBar.tsx
│           ├─ ResultList.tsx
│           ├─ ResultCard.tsx
│           ├─ GroupSection.tsx
│           ├─ SummaryPanel.tsx
│           ├─ CitationChip.tsx
│           └─ BookmarkGrid.tsx
└─ docs/
   ├─ 搜索聚合器-构建方案.md
   └─ 决策与接口变更.md
```

5 数据模型

```
-- 书签
CREATE TABLE bookmark_groups (
  id      INTEGER PRIMARY KEY,
  name    TEXT NOT NULL,
  icon    TEXT,
  sort    INTEGER DEFAULT 0
);

CREATE TABLE bookmarks (
  id          INTEGER PRIMARY KEY,
  url         TEXT NOT NULL UNIQUE,
  title       TEXT,
  favicon     TEXT,
  group_id    INTEGER REFERENCES bookmark_groups(id),
  note        TEXT,
  created_at  INTEGER NOT NULL,
  last_visit  INTEGER,
  visits      INTEGER DEFAULT 0,
  dead        INTEGER DEFAULT 0          -- 0 正常 1 抓取失败
);
CREATE INDEX idx_bookmarks_group ON bookmarks(group_id);

CREATE TABLE bookmark_tags (
  bookmark_id INTEGER REFERENCES bookmarks(id) ON DELETE CASCADE,
  tag         TEXT NOT NULL,
  PRIMARY KEY (bookmark_id, tag)
);

CREATE VIRTUAL TABLE bookmarks_fts USING fts5(
  title, url, note,
  content='bookmarks', content_rowid='id'
);

-- 检索缓存
CREATE TABLE search_cache (
  qkey    TEXT PRIMARY KEY,   -- hash(q|engine|region|page|opts)
  payload TEXT NOT NULL,      -- 已分组的 JSON
  ts      INTEGER NOT NULL,
  ttl     INTEGER DEFAULT 900
);

-- 域名 / URL 信誉
CREATE TABLE url_reputation (
  url_hash TEXT PRIMARY KEY,
  risk     REAL NOT NULL,
  sources  TEXT,              -- "gsb,urlhaus"
  ts       INTEGER NOT NULL
);

-- 页面总结缓存
CREATE TABLE summaries (
  content_hash TEXT PRIMARY KEY,
  url          TEXT,
  title        TEXT,
  text         TEXT,
  model        TEXT,
  ts           INTEGER NOT NULL
);

-- 历史与埋点
CREATE TABLE history (
  id     INTEGER PRIMARY KEY,
  q      TEXT, engine TEXT, region TEXT,
  ts     INTEGER NOT NULL,
  clicks INTEGER DEFAULT 0
);

-- 引擎健康状态（熔断用）
CREATE TABLE engine_health (
  engine      TEXT PRIMARY KEY,
```

```
  fails       INTEGER DEFAULT 0,
  open_until  INTEGER DEFAULT 0,
  last_ok     INTEGER,
  avg_ms      INTEGER
);
```

---

6 API 契约

6.1 搜索

```
POST /api/search
Content-Type: application/json

{
  "q": "数分知识点 g",
  "page": 1,
  "region": null,                  // 可被后缀覆盖
  "opts": { "summarize": true, "exact": false, "safe": true }
}

200 OK
{
  "parsed": {
    "q": "数分知识点",
    "engine": "google",
    "region": "cn",
    "router": "suffix",            // suffix | bang | default
    "expanded": ["数据分析 知识点", "数理统计 知识点"]
  },
  "groups": {
    "strong":  [Result, ...],
    "related": [Result, ...],
    "weak":    [Result, ...]
  },
  "meta": {
    "engines": ["google", "bing", "brave"],
    "cached": false,
    "ms": 1240,
    "filtered": { "risk": 2, "dup": 7 },
    "sid": "s_9f3a"
  }
}

Result = {
  "id": "r_12",
  "url": "https://example.com/a",
  "display_url": "example.com › a",
  "title": "...",
  "snippet": "...",
  "engine": ["google", "brave"],
  "score": 0.78,
  "risk": 0.02,
  "reasons": ["标题命中 3 词", "2 引擎共识", "权威域名"],
  "favicon": "https://example.com/favicon.ico",
  "published": "2026-09-01",
  "read_minutes": 6
}
```

6.2 流式总结

```
GET /api/summarize?sid=s_9f3a&mode=fast|full
Accept: text/event-stream

event: meta   data: {"model":"qwen3:8b","sources":10,"mode":"fast"}
event: cite   data: {"i":1,"title":"...","url":"https://...","domain":"example.com"}
event: token  data: {"t":"根据"}
event: token  data: {"t":"以上材料，"}
event: done   data: {"ms":3120,"tokens":418,"partial":false}
event: error  data: {"code":"LLM_TIMEOUT","retryable":true}
```

- `mode=fast`：只用搜索结果 snippet，约 1.5s 首字。
- `mode=full`：抓正文后再总结，约 3s 首字；前端拿到 `done` 后把快总结替换为完整版并打"已增强"标记。
- 前端只渲染 `cite` 事件里出现过的编号，其它 `[n]` 一律视为无效引用丢弃。

6.3 页面总结

```
POST /api/page/summarize
{ "url": "https://...", "mode": "auto" }

200 OK
{
  "url": "...", "title": "...", "byline": "...", "published": "...",
  "word_count": 4210, "read_minutes": 12, "content_hash": "sha1:...",
  "summary": {
    "one_liner": "……",
    "bullets": ["…", "…"],
    "key_data": [{"label":"…","value":"…"}],
    "caveats": ["…"]
  },
  "cached": false, "model": "qwen3:8b"
}
```

6.4 书签

```
GET    /api/bookmarks?group=&q=
POST   /api/bookmarks            { url, title?, group_id?, tags?[] }
PATCH  /api/bookmarks/{id}       { title?, group_id?, note?, sort? }
DELETE /api/bookmarks/{id}
POST   /api/bookmarks/import     multipart/form-data  (Netscape HTML)
GET    /api/bookmarks/export                 -> text/html (Netscape)
POST   /api/bookmarks/check                  -> 批量死链检测
```

6.5 风险中转

```
GET /go?u=<base64url>&sig=<hmac>
  200  text/html
  # 展示目标域名、域名年龄、风险分、命中来源，"仍要访问"才跳转
  # sig 防篡改，避免被当成开放重定向
```

---

7 核心模块设计

7.1 查询解析器

规则优先级：bang > 后缀短码 > 默认。

```ts
const ENGINES: Record<string, string> = {
  g: 'google',   b: 'bing',   d: 'duckduckgo', ddg: 'duckduckgo',
  br: 'brave',   yt: 'youtube', gh: 'github',  ar: 'arxiv',
  sc: 'scholar', w: 'wikipedia',
  zh: 'cn', cn: 'cn', jp: 'jp', us: 'us', en: 'en',
};

export function parseQuery(raw: string) {
  const q = raw.normalize('NFKC').trim();

  // 1) bang：无歧义优先
  const bang = q.match(/\s*!([a-z]{1,3})$/i);
  if (bang && ENGINES[bang[1].toLowerCase()])
    return { q: q.slice(0, bang.index).trim(),
             engine: ENGINES[bang[1].toLowerCase()], router: 'bang' };

  // 2) 后缀短码：前面必须有空白，且必须是纯 ASCII 字母
  const m = q.match(/\s+([a-z]{1,3})$/i);
  if (m && ENGINES[m[1].toLowerCase()])
    return { q: q.slice(0, m.index).trim(),
             engine: ENGINES[m[1].toLowerCase()], router: 'suffix' };

  return { q, engine: 'default', router: 'default' };
}
```

护栏：

8. 短码前必须是空白（含全角空格 `\u3000`），`C g` 这类单字母前置于数字/单字母时降级为普通查询。
9. 引号内不解析：`"sort g"` 保持原样。
10. `!g` 与 `g` 双写法并存，bang 给习惯 DuckDuckGo 的用户，后缀给中文输入场景。

后缀映射表（可在 `config.py` 里扩展为 JSON 配置）：

后缀	引擎 / 地区	说明
`g`	Google	主用
`b` / `d`	Bing / DuckDuckGo	备用
`br`	Brave	独立索引
`yt gh ar sc w`	YouTube / GitHub / arXiv / Scholar / Wikipedia	垂类
`zh cn`	中文区（百度 + 360 + 搜狗）	地区切
`jp us`	对应 locale 引擎集	地区切
无	默认多引擎聚合	主力路径

7.2 检索聚合

```
for engine in selected_engines:      # 并发，asyncio.gather
    if breaker_open(engine): skip
    fetch with timeout=2.0s
    on success -> record ok, reset fails
    on fail    -> fails += 1; fails >= 3 -> open 3600s
```

- 主路径：`SearXNG /search?q=...&format=json&engines=...`
- 回退链：SearXNG → Brave API → Serper，任一成功即停。
- 引擎子集由解析出的 `engine`/`region` 决定；默认取 3–5 个健康引擎，避免结果同质化。

7.3 去重与归一化

```
def canonical(url: str) -> str:
    u = urlsplit(url.strip())
    q = [(k, v) for k, v in parse_qsl(u.query)
         if k.lower() not in TRACKING_PARAMS]        # utm_*, fbclid, gclid, spm
    path = u.path.rstrip('/') or '/'
    return urlunsplit((u.scheme.lower(), u.netloc.lower(), path,
                       urlencode(sorted(q)), ''))
```

二级去重：标题 simhash（64 位，汉明距离 ≤ 3）判同稿，保留权威度最高的那条并合并 `engine` 列表。

7.4 安全过滤

三源黑名单（本地缓存 6h）

源	用途	接入要点
Google Safe Browsing Lookup v4	恶意/钓鱼/垃圾站	免费，默认配额够个人用；**仅限非商用**，商用换 Web Risk
URLhaus	恶意软件分发	**2025-06-30 起必须带 Auth-Key**
OpenPhish Community	钓鱼 URL	免费无 key，12 小时更新

启发式风险分

```
risk = (0.45 * feed_hit          # 黑名单命中 0/1
      + 0.20 * age_penalty       # <90 天 = 1.0 (无 RDAP 记录 0.8) , <1 年 = 0.5
      + 0.15 * tld_penalty       # .zip .top .xyz .tk .icu .cyou → 1.0
      + 0.10 * url_shape         # IP 直连 / 超长随机串 / >4 层子域 / user@host
      + 0.10 * homoglyph)        # punycode(xn--) / 形近字(0-O, rn-m, 1-l)
```

处置：`risk ≥ 0.6` 折叠隐藏；`0.35 ≤ risk < 0.6` 保留但跳转走 `/go` 中转确认页；`< 0.35` 放行。

质量分（独立于风险分）：标题堆叠关键词（关键词密度 > 25%）、正文 < 200 字、域名是已知聚合站、与查询语义漂移（出现"加微信/客服/限时优惠/下载 APP"等营销词且查询无相关意图）→ 直接进弱相关或丢弃。

> *边界说明：黑名单是概率保证，时延从几分钟到几小时。域名年龄 + 跳转确认两层兜底是必需的，不能只依赖 feed。*

7.5 打分与三档分组

特征	计算	权重
词法	BM25(query, title×2 + snippet)，jieba 分词	0.30
语义	cos(emb(q), emb(title+snippet))	0.30
引擎共识	出现在 n 个引擎 top10 → min(n,3)/3	0.15
权威度	域名白名单（.gov/.edu/官方文档/Wikipedia/GitHub/期刊）	0.10
时效	查询含时间意图时，按 published 距今衰减	0.05
负向	风险分 ×−0.15、spam 特征、内容农场、标题党	−0.20 ~ 0

```
def group_results(results, n_strong=10, n_mid=10, n_weak=5):
    for r in results:
        r.score = (0.30 * norm_bm25(r)
                 + 0.30 * r.sem
                 + 0.15 * r.consensus
                 + 0.10 * r.authority
                 + 0.05 * r.freshness
                 - 0.15 * r.risk
                 - 0.10 * r.spam)
    s = minmax_scale([r.score for r in results])
    strong  = [r for r, x in zip(results, s) if x >= 0.62][:n_strong]
    mid     = [r for r, x in zip(results, s) if 0.35 <= x < 0.62][:n_mid]
    weak    = [r for r, x in zip(results, s) if x < 0.35][:n_weak]
    return strong, mid, weak
```

双层机制（精确模式）：公式先秒出三档 → 后端把 20 条 `{title, snippet, domain}` 打包成一次 LLM batch 判定（约 500 token）→ 前端显示"正在校准..." → 判定回来重排并给每条附一句"为什么"。成本可忽略，准确率明显更稳。

空组不渲染；弱相关默认折叠，展开后才加载 favicon。

7.6 总结管线（RAG-lite）

```
1. 取 top 8~12 条结果
2. 并发抓正文（6~8 并发，2.5s 超时，失败降级用 snippet）
3. 正文提取：trafilatura → 失败走 readability → 再失败走 r.jina.ai
4. 每篇截断 2000~3000 token，保留标题/URL/发布日期
5. 组装 prompt，强制引用编号
6. SSE 流式输出；同步维护 citations 数组
7. 落库：summaries(content_hash) / search_cache(qkey)
```

延迟预算：检索 0.8–2s → 抓正文 1.5–2.5s → 首 token 2.5–4s。

优化手段：`mode=fast` 先出结果、再升级；正文抓取按域名做优先级（权威域名先抓）；命中缓存直接复用。

7.7 网页总结管线

```
URL → 抓取（正常 UA、遵守 robots.txt）
    → JS 站点用 Playwright 兜底
    → 正文提取 → 按标题层级分块
    → ≤30k token 直接总结；超出走 map-reduce（块摘要 → 合并 → 冲突消解）
    → 固定结构输出
```

输出结构固定为五段：一句话结论 / 5 条要点 / 关键数据表 / 争议与不确定点 / 原文锚点。

失败场景（付费墙、反爬、纯图片）→ 明确标注"仅基于摘要生成，置信度低"。

7.8 书签一键添加（四条通道）

通道	实现	体验	代价
Chrome 扩展	`chrome.bookmarks` + `contextMenus` + `activeTab`	桌面最顺，真一键	需要加载/上架
Bookmarklet	拖到书签栏的一行 JS，POST 当前 URL	任意浏览器通用	两次点击，移动端不可用
PWA share_target	manifest 声明 share target	移动端最顺	仅装 PWA 的机型

通道	实现	体验	代价
快捷键 / 剪贴板	`Ctrl+Shift+D` 存剪贴板 URL	配合桌面壳最爽	需容器

书签页还应有：分组网格、拖拽排序、数字键快捷启动（1–9）、自动 favicon、死链快照、搜索过滤、**Netscape HTML 导入导出**（与浏览器互迁）。

---

8 前端页面设计

8.1 页面与组件树

```
/search 页
├─ SearchBar              大输入框，自动补全（本地历史 + SearXNG autocomplete）
├─ HotKeywords            今日热词 / 最近搜索
└─ QuickBookmarks         前 8 个书签图标

/r 结果页
├─ ResultToolbar          引擎切换、地区、安全开关、精确模式开关
├─ ResultList
│  ├─ GroupSection "强相关"  （默认展开）
│  │  └─ ResultCard × N    标题/域名/摘要/命中词高亮/风险角标/[总结]
│  ├─ GroupSection "相关"    （默认展开）
│  └─ GroupSection "弱相关"  （默认折叠）
└─ SummaryPanel            sticky 右侧栏，320–420px，可折叠可拖宽
   ├─ SummarySkeleton      骨架 + "正在检索…"
   ├─ SummaryBody          流式正文，[n] 可点，点击滚动并高亮左侧对应卡片
   └─ SummaryFooter        模型、耗时、来源数、"重新生成"

/bookmarks 页
├─ BookmarkGrid           分组标签页 + 卡片网格
├─ AddButton              一键添加（粘贴 URL / 扩展 / bookmarklet 提示）
└─ ImportExport
```

8.2 关键交互

- 布局：结果区 `flex-1`，侧栏 `w-[380px] shrink-0 lg:block`；`< lg` 断点下降级为底部抽屉（50% 高，可上拉）。
- 侧栏不遮挡：`position: sticky; top: 72px; max-height: calc(100vh - 88px); overflow:auto`。
- 状态机：`idle → searching → retrieving → streaming → done | partial | error`，每个态都有对应文案，不出现空白。
- 快捷键：`/` 聚焦搜索，`j/k` 上下选，`Enter` 打开，`b` 收藏，`s` 总结，`Esc` 关侧栏，`g s` 回搜索页。
- 引用联动：`[3]` → 左侧第 3 条卡片 `scrollIntoView({block:'center'})` + 1.5s 高亮描边。

8.3 移动端

PWA + `share_target`：从任意 App 分享链接 → 直接进书签收集箱；结果页侧栏变抽屉；输入框吸底。

9 提示词模板

9.1 总结（system）

```
你是搜索结果的整理器。只依据用户提供的材料回答，不使用材料以外的知识。
规则：
1. 每个结论句末尾标注来源编号，格式 [1][2]。
2. 材料不足以支撑的结论不要写；如果整体材料不足，直接输出"材料不足：……"并说明缺什么。
3. 不写客套话、不写"根据您提供的材料"这类开场。
4. 中文回答，术语保留原文。
5. 输出结构：一段 3–5 句的结论 → 换行 → 3–5 条要点（每条一行，- 开头）→ 如有分歧，最后一行以"分歧："开头。
```

9.2 总结（user）

```
问题：{query}

材料：
[1] 标题：{title} | 域名：{domain} | 链接：{url}
{content}

[2] ...
```

9.3 相关性判定（batch）

```
把下列候选网页按与查询的相关程度分为三档。
查询：{query}

候选：
1. {title} | {domain} | {snippet}
...

只输出 JSON：
{"1":"strong|related|weak","2":"...","reason":{"1":"不超过 12 字的理由"}}
```

10 配置

10.1 docker-compose.yml

```
services:
  searxng:
    image: searxng/searxng:latest
    container_name: cs-searxng
    ports: ["8888:8080"]
    volumes: ["./searxng:/etc/searxng"]
    environment:
      - SEARXNG_BASE_URL=http://127.0.0.1:8888/
      - SEARXNG_SECRET=CHANGE_ME
    restart: unless-stopped
    networks: [cs]

  api:
    build: ./apps/api
    container_name: cs-api
    env_file: .env
    ports: ["8000:8000"]
    depends_on: [searxng]
    volumes: ["./data:/data"]
    restart: unless-stopped
    networks: [cs]

networks:
  cs:
```

10.2 searxng/settings.yml（关键项）

```
use_default_settings: true
server:
  limiter: false
  image_proxy: true
  secret_key: "CHANGE_ME"
search:
  safe_search: 1
  formats: [html, json]      # json 必须显式开
  autocomplete: ""
```

10.3 .env.example

```
SEARXNG_URL=http://127.0.0.1:8888
BRAVE_API_KEY=
SERPER_API_KEY=
GSB_API_KEY=
URLHAUS_AUTH_KEY=
OPENPHISH_FEED=https://openphish.com/feed.txt
LLM_BASE_URL=http://127.0.0.1:11434/v1
LLM_API_KEY=ollama
LLM_MODEL=qwen3:8b
EMBED_MODEL=bge-m3
DB_PATH=/data/cs.db
CACHE_TTL=900
FETCH_CONCURRENCY=8
FETCH_TIMEOUT=2.5
```

---

11 数据源与成本

来源	类型	免费额度	之后	主要坑
SearXNG 自建	元搜索聚合	无限	0	上游引擎（DDG/Qwant/百度/搜狗）会 CAPTCHA 与限流，必须多引擎池 + 熔断 + 代理

来源	类型	免费额度	之后	主要坑
Brave Search API	独立索引（非转售）	2,000 次/月，1 QPS	约 $3–5 / 千次	最省心的商业 API
Google CSE JSON API	Google 官方	100 次/天	$5 / 千次，上限 1 万/天	结果受 CSE 配置影响，有展示条款
Serper	Google SERP 转售	2,500 次（一次性）	$50 / 5 万，大包低至 $0.3 / 千	第三方转售，价格最便宜
Tavily / Exa	面向 AI 的检索	Tavily 1,000 credits/月	按量	返回已清洗内容，适合总结，不适合"给人看的排名"
Bing Web Search API	—	—	—	2025-08-11 已停服，不可用

组合策略：默认 SearXNG → 引擎失败或强时效查询回退 Brave / Serper → 正文自建 trafilatura，失败回退 r.jina.ai。日常零成本，关键时刻不塌。

参考链接：

- Brave 定价：https://requestly.com/api-explorer/brave-search/
- Google CSE 限额：https://developers.google.cn/custom-search/v1/overview
- Serper 价格：https://serper.dev
- Tavily 定价：https://www.tavily.com/pricing
- Safe Browsing 使用限额：https://developers.google.cn/safe-browsing/v4/usage-limits
- URLhaus 需要 Auth-Key：https://abuse.ch/blog/community-first/
- OpenPhish Feed：https://openphish.com/phishing_feeds.html

---

12 里程碑与验收

里程碑	内容	工期	验收标准
M0 骨架	docker-compose 起 SearXNG、FastAPI 空壳、搜索框 + 裸结果列表	0.5 天	`curl /api/search?q=test` 返回 10 条
M1 结果页	多引擎并发、去重、打分、三板块渲染	1 天	10 组查询，弱相关栏垃圾率 < 20%
M2 侧栏总结	SSE 流式、引用锚点、卡片联动	1 天	首字 < 4s，引用编号 100% 可跳转
M3 书签台	CRUD、分组、一键添加（bookmarklet 先行）、导入导出	1 天	从 Chrome 导入 500 条不丢字段
M4 安全过滤	三源黑名单、风险分、`/go` 中转	1 天	20 条已知钓鱼样本 100% 拦截
M5 后缀路由	后缀 + bang + 护栏	0.5 天	数分知识点 `g` 正确走 Google，`C g` 不误触发
M6 网页总结	提取、分块、map-reduce、五段结构	1 天	10k 字文章摘要 < 8s，锚点可回原文
M7 打磨	缓存、快捷键、移动端、暗色、错误态	1.5–2 天	断网 / LLM 超时 / 引擎全挂都有体面降级
M8 扩展（可选）	一键加书签、右键总结、页面拦截	1.5 天	任意页右键 1 次入书签

里程碑	内容	工期	验收标准
M9 桌面包（可选）	Tauri 壳 + WebContentsView	3–4 天	内置 adblock，侧栏总结非本系统页面

合计：主线 7–10 个工作日；含扩展 12–15 天；含桌面包 20–25 天。

---

13 风险与缓解

风险	影响	缓解
SearXNG 上游被封	结果是空或质量骤降	多引擎池 + 自动熔断（失败 3 次冷却 1h）+ Brave/Serper 兜底 + 结果页显式提示"部分引擎不可用"
中文短查询语义弱	三档切得随机	查询扩展（同义/上位词）+ 提高词法权重 + LLM 复核
LLM 引用幻觉	结论不可追	强制 `[n]`、后端校验编号在集合内、前端只渲染 cite 事件里的编号
抓正文超时	首字延迟飙升	6–8 并发、2.5s 超时、fast/full 双档、权威域名优先
黑名单时延	漏放新钓鱼站	域名年龄 + `/go` 中转确认 + 用户举报入口
一键加书签受限	体验落差	三通道并行，桌面主推扩展
弱相关栏熵增	变成垃圾场	默认折叠、最多 5 条、域名一键屏蔽
第三方条款	公开服务时受限	CSE 有展示要求、Serper 属转售、abuse.ch 有 ToU；自用无碍，公开前逐项核对

---

14 待拍板事项

编号	问题	选项	默认建议
Q1	交付形态	A 纯 PWA / B PWA+扩展 / C 桌面容器	B（先 A 的 M0–M2，再补扩展）
Q2	LLM 走本地还是 API	本地 Ollama / 云端 API	本地（成本 0、离线可用），深度模式再切云端
Q3	是否多用户	单人 / 家庭共享 / 公开	单人，schema 预留 `user_id`
Q4	部署位置	本机 / 局域网 VPS / 公网	本机 + 局域网
Q5	是否要移动端	是 / 否	是（PWA 顺带，成本很低）
Q6	引擎偏好	以 Google 为主 / 三引擎均衡	三引擎均衡，避免单一来源审查与偏差

附录 A 关键代码骨架

A.1 多引擎并发与熔断

```
async def gather_results(q: str, engines: list[str], page: int) -> list[dict]:
    async with httpx.AsyncClient(timeout=2.0) as cli:
        tasks = [fetch_one(cli, q, e, page) for e in engines if not breaker.is_open(e)]
        done = await asyncio.gather(*tasks, return_exceptions=True)
    out = []
    for e, r in zip(engines, done):
        if isinstance(r, Exception):
            breaker.fail(e)
            continue
        breaker.ok(e)
        out.extend(r)
    return out
```

A.2 SSE 端点

```
@router.get("/api/summarize")
async def summarize(sid: str, mode: str = "fast"):
    ctx = cache.get_context(sid)
    async def gen():
        yield sse("meta", {"model": settings.LLM_MODEL, "sources": len(ctx.sources), "mode": mode})
        for i, s in enumerate(ctx.sources, 1):
            yield sse("cite", {"i": i, "title": s.title, "url": s.url, "domain": s.domain})
        async for tok in llm.stream(ctx.prompt(mode)):
            yield sse("token", {"t": tok})
        yield sse("done", {"ms": ctx.elapsed_ms(), "partial": mode == "fast"})
    return StreamingResponse(gen(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})
```

A.3 标题 simhash

```
def simhash(text: str, bits: int = 64) -> int:
    v = [0] * bits
    for w in jieba.cut(text):
        h = hash64(w)
        for i in range(bits):
            v[i] += 1 if (h >> i) & 1 else -1
    return sum(1 << i for i in range(bits) if v[i] > 0)
```

A.4 书签一键添加（bookmarklet）

```
javascript:(()=>{const t=document.title,u=location.href;
fetch('http://127.0.0.1:8000/api/bookmarks',{method:'POST',
headers:{'Content-Type':'application/json'},
body:JSON.stringify({url:u,title:t})}).then(()=>alert('已收藏'))})()
```

附录 B 术语

词	含义
Bang	DuckDuckGo 风格的前缀/后缀指令，!g 表示走 Google
RAG-lite	不建向量库，直接把检索到的正文塞进上下文做总结
熔断	某引擎连续失败后短期跳过，避免拖慢整体
simhash	局部敏感哈希，用于近似重复检测
qkey	查询缓存键，由查询 + 引擎集 + 页码 + 选项哈希而成

文档结束。可直接作为开发交接件使用；接口变更请同步到 `docs/决策与接口变更.md`。